NoSQL Systems

MongoDB - II

"A true NoSQL document-oriented database system"

Vinu Venugopal
ScaDS.ai Lab, IIIT Bangalore

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- DELETE
- BULK WRITE

MongoDB CRUD Operations

CRUD Operations:

- **CREATE**
- READ
- UPDATE
- DELETE
- BULK WRITE

- Create or insert operations add new documents to a collection
- If the collection does not currently exist, insert operations will create the collection

Methods:

```
db.collection.insertOne()
db.collection.insertMany()
```

```
db.users.insertOne(          ← collection
  {
    name: "sue",             ← field: value
    age: 26,                 ← field: value   } document
    status: "pending"        ← field: value
  }
)
```

MongoDB CRUD Operations

CRUD Operations:

- **CREATE**
- READ
- UPDATE
- DELETE
- BULK WRITE

- Create or insert operations add new documents to a collection

- If the collection does not currently exist, insert operations will create the collection

Methods:
`db.collection.insertOne()`
`db.collection.insertMany()`

```
const doc1 = { "name": "basketball", "category": "sports", "quantity": 20, "reviews": [] };
const doc2 = { "name": "football", "category": "sports", "quantity": 30, "reviews": [] };

db.itemsCollection.insertMany([doc1, doc2])
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- **READ**
- UPDATE
- DELETE
- BULK WRITE

- Retrieves documents from a collection

Methods:
`db.collection.find()`

```
db.users.find(                          ← collection
   { age: { $gt: 18 } },                ← query criteria
   { name: 1, address: 1 }              ← projection
).limit(5)                              ← cursor modifier
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- **UPDATE**
- DELETE
- BULK WRITE

- Modify existing documents in a collection

Methods:

```
db.collection.updateOne()
db.collection.updateMany()
db.collection.replaceOne()
```

```
db.users.updateMany(                    ← collection
  { age: { $lt: 18 } },                 ← update filter
  { $set: { status: "reject" } }        ← update action
)
```

- **ReplaceOne**: Replaces a single document within the collection based on the filter.

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- **DELETE**
- BULK WRITE

- Remove documents from a collection

Methods:

```
db.collection.deleteOne()
db.collection.deleteMany()
```

```
db.users.deleteMany(          ← collection
  { status: "reject" }        ← delete filter
)
```

MongoDB CRUD Operations

CRUD Operations:

- CREATE
- READ
- UPDATE
- DELETE
- **BULK WRITE**

- Provides the ability to perform bulk insert, update and remove operations

Methods:
`db.collection.bulkWrite()`

Supports the following write operations:
`insertOne`
`updateOne`
`updateMany`
`replaceOne`
`deleteOne`
`deleteMany`

```
db.characters.bulkWrite(
      [ { insertOne :
            { "document" :
               {"_id" : 4, "char" : "Dithras",
                 "class" : "barbarian", "lvl" : 4 }
            }
         },
         { insertOne :
            { "document" :
               {"_id" : 5, "char" : "Taeln",
                "class" : "fighter", "lvl" : 3 }
            }
         },
         { updateOne :
            {  "filter" : { "char" : "Eldon" },
               "update" : { $set : {
                     "status" : "Critical Injury" } }
            }
         },
         { deleteOne :
            { "filter" : { "char" : "Brisbane"} }
         },
         { replaceOne :
            {  "filter" : { "char" : "Meldane" },
               "replacement" : { "char" : "Tanys",
                          "class" : "oracle", "lvl" : 4 }
            }
         }] )
```

Bulkloading Large Files: `mongoimport`

- `mongoimport` supports JSON, CSV and TSV formats (all of which are ASCII-based) for bulk-loading large data volumes into a MongoDB collection

- From the Linux command line, use:

```
$mongoimport -- db <database-name> -u <groupname> -p <password>
            -- collection keywords
            -- fields docid, term, score
            -- type tsv –file <path-to-keywords.tsv>
```

- `mongoexport` is the counterpart of dumping a MongoDB collection into a file.

It writes (binary) BSON format back into ASCII/UTF format.

Joins

MongoDB does **not** support joins (Versions < 3.2)

- Data is denormalized , or stored with related data in documents

- MongoDB 3.2 introduces $lookup pipeline stage to perform a left outer join to an unsharded collection in the same database.

Example: A single equality Join with $lookup

```
db.orders.insert([
   { "_id" : 1, "item" : "almonds", "price" : 12, "quantity" : 2 },
   { "_id" : 2, "item" : "pecans", "price" : 20, "quantity" : 1 },
   { "_id" : 3  }
])
```

```
db.inventory.insert([
   { "_id" : 1, "sku" : "almonds", description: "product 1", "instock" : 120 },
   { "_id" : 2, "sku" : "bread", description: "product 2", "instock" : 80 },
   { "_id" : 3, "sku" : "cashews", description: "product 3", "instock" : 60 },
   { "_id" : 4, "sku" : "pecans", description: "product 4", "instock" : 70 },
   { "_id" : 5, "sku": null, description: "Incomplete" },
   { "_id" : 6 }
])
```

Joins

MongoDB does not support joins (Versions < 3.2)

- Data is denormalized , or stored with related data in documents

- MongoDB 3.2 introduces $lookup pipeline stage to perform a left outer join to an unsharded collection in the same database.

Example: A single equality Join with $lookup (via creating a data processing pipeline using aggregate.)

```
db.orders.aggregate([
   {
     $lookup:
       {
         from: "inventory",
         localField: "item",
         foreignField: "sku",
         as: "inventory_docs"
       }
  }
])
```

Syntax:
from: <foreign collection>,
localField: <field from local collection's documents>,
foreignField: <field from foreign collection's documents
as: <output array field>

Indexes

- **Automatically maintains an index on the primary key** (the "_id" field) of objects in a collection
- Supports arbitrary **secondary** indexes.
- Supports indexes on **any field or sub-field** of the documents in a collection
- Indexes use a B-tree data structure

- **Example**:

```
{
  "_id": ObjectId("570c04a4ad233577f97dc459"),
  "score": 1034,
  "location": { state: "NY", city: "New York" }
}
```

```
db.records.createIndex( { "location.state": 1 } )
```

Index type
1 (ascending) or -1 (descending)

Supports queries:

```
db.records.find( { "location.state": "CA" } )
db.records.find( { "location.city": "Albany", "location.state": "NY" } )
```

- Range queries will also utilize these index structures.

Indexes

Compound Indexes

- supports *compound indexes*, where a single index structure holds references to multiple fields
- **Example**:

```
{
 "_id": ObjectId(...),
 "item": "Banana",
 "category": ["food", "produce", "grocery"],
 "location": "4th Street Store",
 "stock": 4,
 "type": "cases"
}
```

- Ascending index on the *item* and *stock* fields:

```
db.products.createIndex( { "item": 1, "stock": 1 } )
```

Supports queries:

```
db.products.find( { item: "Banana" } )
db.products.find( { item: "Banana", stock: { $gt: 5 } } )
```

Use Indexes to Sort Query Results

```
db.data.createIndex( { a:1, b: 1, c: 1, d: 1 } )
```

Prefixes of the index:

```
{ a: 1 }
{ a: 1, b: 1 }
{ a: 1, b: 1, c: 1 }
```

Sort operations that take advantage of the index prefixes:

Use Indexes to Sort Query Results

db.data.createIndex({ a:1, b: 1, c: 1, d: 1 })

Prefixes of the index:

{ a: 1 }

{ a: 1, b: 1 }

{ a: 1, b: 1, c: 1 }

Sort operations that take advantage of the index prefixes:

Example	Index Prefix
db.data.find().sort({ a: 1 })	{ a: 1 }
db.data.find().sort({ a: -1 })	{ a: 1 }
db.data.find().sort({ a: 1, b: 1 })	{ a: 1, b: 1 }
db.data.find().sort({ a: -1, b: -1 })	{ a: 1, b: 1 }
db.data.find().sort({ a: 1, b: 1, c: 1 })	{ a: 1, b: 1, c: 1 }
db.data.find({ a: { $gt: 4 } }).sort({ a: 1, b: 1 })	{ a: 1, b: 1 }

Mapping of SQL concepts to MongoDB

SQL Terms/Concepts	MongoDB Functions
GROUP BY	$group
HAVING	$match
SELECT	$project
ORDER BY	$sort
LIMT	$limit
SUM()	$sum
COUNT()	Only indirectly via: $sum:1

Aggregation

- Aggregate documents from a single collection **using available built-in functions**:

Examples:

```
db.collection.distinct()
db.collection.count()
```

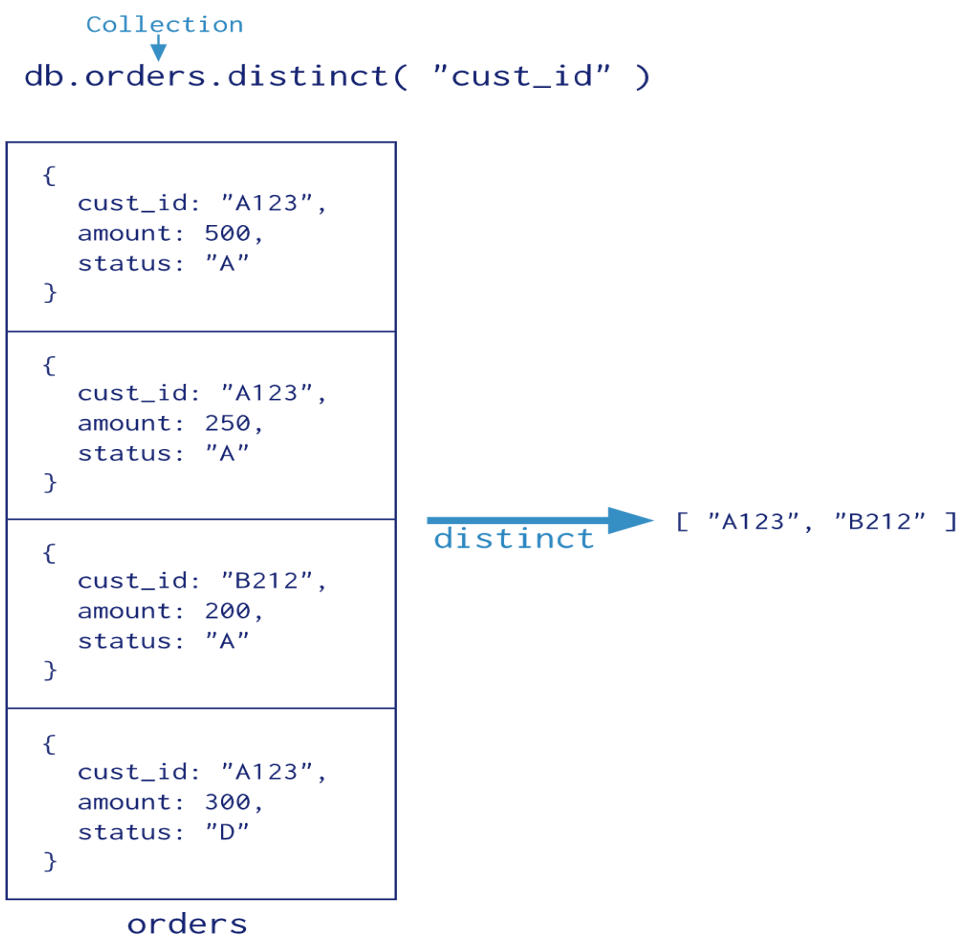
To perform aggregation operations, you can use:

Aggregation pipelines, which are the preferred method for performing aggregations.

Single purpose aggregation methods, which are simple but lack the capabilities of an aggregation pipeline.

Single purpose aggregation methods:

db.collection.estimatedDocumentCount()
db.collection.count()
db.collection.distinct()



Aggregation

- Perform the same count operation using “aggregate” operator:

```
db.keywords.aggregate(
  [
      {$group: { _id : null, counts : {$sum : 1}}}
  ])
```

- Multiple operations can be combined into a “**Pipeline**”

```
db.collection.aggregate( [ { <stage> }, ... ] )
```

Aggregation

- Perform the same count operation using “aggregate” operator:

```
db.keywords.aggregate(
  [
      {$group: { _id : null, counts : {$sum : 1}}}
  ])
```

- Multiple operations can be combined into a “**Pipeline**”

```
db.collection.aggregate( [ { <stage> }, ... ] )
```

```
{ $group: { _id: <expression>, // Group By Expression
            <field1>: { <accumulator1> : <expression1> },
            ... }
 }
```

The _id expression specifies the group key. If you specify an _id value of null, or any other constant value, the $group stage returns a single document that aggregates values across all of the input documents.

https://www.mongodb.com/docs/manual/reference/operator/aggregation/group/

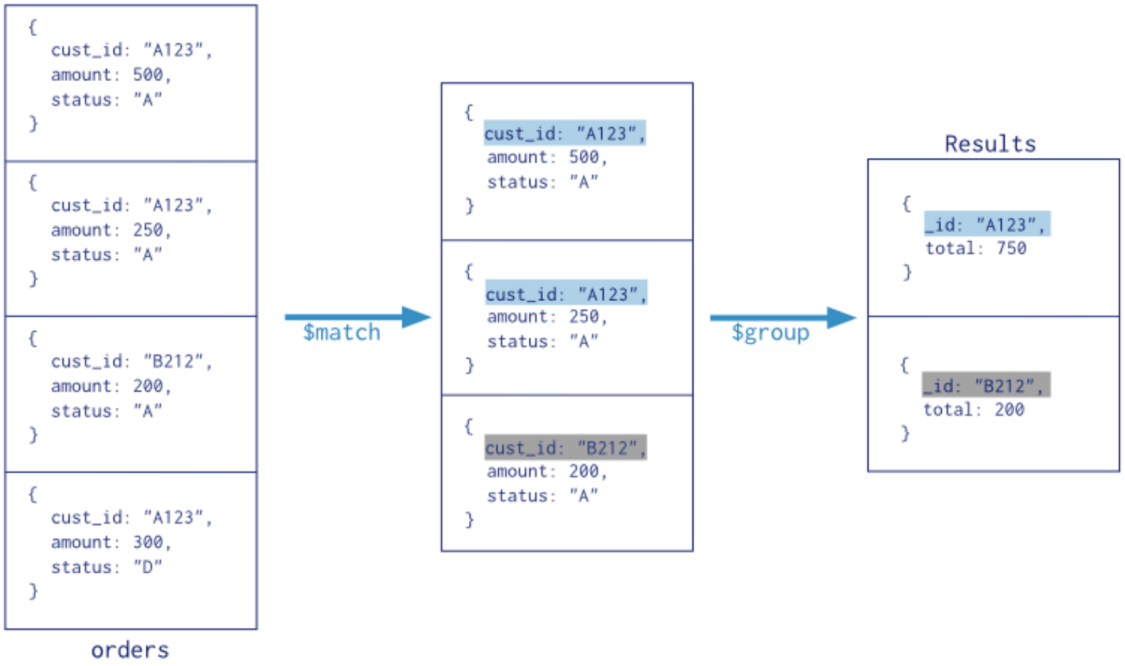
Aggregation Pipelines

Example:

```
db.orders.aggregate([
   { $match: { status: "A" } },
   { $group: { _id: "$cust_id", total: { $sum: "$amount" } } }
])
```

2 stages

- $match
- $group



Aggregation Pipelines

Comparison to GROUP-BY in SQL

```
SELECT COUNT(*) AS count
FROM orders;
```

```
db.orders.aggregate( [
        {$group: {_id:null, count:{$sum:1}}}
] )
```

```
SELECT cust_id, SUM(price) AS total
FROM orders
GROUP BY cust_id ORDER BY total;
```

```
db.orders.aggregate( [
{$group: {_id: "$cust_id" , total: {$sum: "$price"}}},
{$sort: {total:1}}
] )
```

Aggregation Pipelines

Comparison to GROUP-BY in SQL

```
SELECT cust_id, SUM(price) as total
FROM orders
WHERE status = 'A’
GROUP BY cust_id
HAVING total > 250;
```

See: http://docs.mongodb.org/manual/reference/ sql-aggregation-comparison/ for more GROUP-BY examples.

Aggregation Pipelines

Comparison to GROUP-BY in SQL

```
SELECT cust_id, SUM(price) as total
FROM orders
WHERE status = 'A’
GROUP BY cust_id
HAVING total > 250;
```

```
db.orders.aggregate([
{$match:{status:'A'}},
{$group:{_id: "$cust_id" ,
total:{$sum:"$price"} }},
{$match:{total:{$gt: 250}}}
] )
```

See: http://docs.mongodb.org/manual/reference/ sql-aggregation-comparison/ for more GROUP-BY examples.

More Operators

Pipeline operators:

- $project, $match, $group, $limit, $sort, $skip

Expression/group operators:

- $sum, $min, $max, $avg, $first, $last

Boolean operators:

- $and, $or, $not

Set operators:

- $setEquals, $setIntersection, $setDifference, $setUnion

Comparison operators:

- $cmp, $eq, $gt, $lt

Arithmetic operators:

- $add, $divide, $mod, $multiply, $subtract

String operators:

- $concat, $toLower, $substr

Conditional operators:

- $cond, $ifNull

See https://docs.mongodb.com/manual/reference/operator/aggregation/ for the complete list

Notes on Aggregation Pipelines

- The $match and $sort pipeline operators can **take advantage of an index** when they **occur at the beginning** of the pipeline

- **Early filtering:** If an aggregation operation requires only a subset of the the data in a collection, the $match, $limit and $skip (skips the first n documents passed to it by the pipeline) operators can be used to restrict the documents that enter at the beginning of the pipeline.

- **When placed at the beginning of a pipeline**, $match operations **may use suitable indexes** to scan only the matching documents in a collection.

Aggregation

Using Map-reduce function

- 2 phases:
  - a ***map*** stage that processes each document and *emits* one or more objects for each input document
  - ***reduce*** phase that combines the output of the map operation
- uses **custom JavaScript functions** to perform the map and reduce operation

Collection

```
db.orders.mapReduce(
        map      →   function() { emit( this.cust_id, this.amount ); },
        reduce   →   function(key, values) { return Array.sum( values ) },
                     {
        query    →     query: { status: "A" },
        output   →     out: "order_totals"
                     }
                 )
```

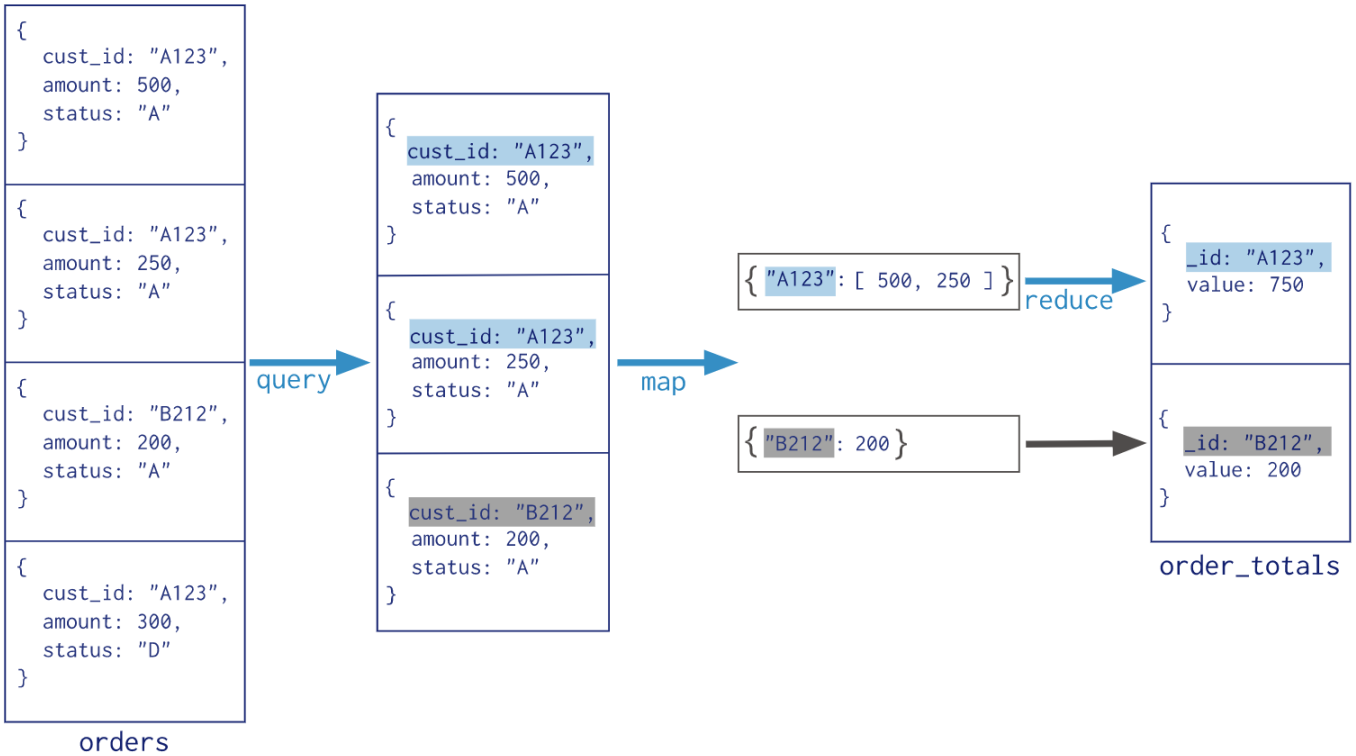
Specifies the selection criteria using query operators

Specifies where to output the result of the map-reduce operation (inline or to a collection)

Aggregation

Collection

```
db.orders.mapReduce(
        map      →  function() { emit( this.cust_id, this.amount ); },
        reduce   →  function(key, values) { return Array.sum( values ) },
                    {
        query    →    query: { status: "A" },
        output   →    out: "order_totals"
                    }
                )
```



Aggregation: mapReduce

- Provides support for MapReduce functions implemented in JavaScript

```
db.runCommand(
              {
                mapReduce: <collection>,
                map: <function>,
                reduce: <function>,
                finalize: <function>,
                out: <output>,
                query: <document>,
                sort: <document>,
                limit: <number>,
                scope: <document>,
                jsMode: <boolean>,
                verbose: <boolean>,
                bypassDocumentValidation: <boolean>,
                collation: <document>,
                writeConcern: <document>
              }
            )
```

Implementation of the Map and Reduce functions

Additional filtering, sorting, size limitations of documents in the input collection

https://docs.mongodb.com/manual/reference/command/mapReduce/#dbcmd.mapReduce

Aggregation: mapReduce

- Provides support for MapReduce functions implemented in JavaScript

```
var mapFunction = function() { ... };
var reduceFunction = function(key, values) { ... };

db.runCommand(
               {
                 mapReduce: <input-collection>,
                 map: mapFunction,
                 reduce: reduceFunction,
                 out: { merge: <output-collection> },
                 query: <query>
               }
             )
```

https://docs.mongodb.com/manual/reference/command/mapReduce/#dbcmd.mapReduce

Aggregation: mapReduce

- The **Map function** must implement the following signature:

```
function() {
  ...
   emit(key, value);
}
```

- The Map function is **called once for each document** in the input collections
- Reference the current documents as "**this**" within the function
- Should **not** access the database for any reason
- **Example**:

```
function() {
    if (this.status == 'A')
        emit(this.cust_id, 1);
}

function() {
    this.items.forEach(function(item){ emit(item.sku, 1); });
}
```

Aggregation: mapReduce

- The **Reduce function** must implement the following signature:

```
function(key, values) {
  ...
 return result;
}
```

- Should **not** access the database, even to perform read operations
- Can access the variables defined in the **`scope`** parameter
- Will not call the reduce function for a key that has only a single value
- The "value" should be always an array
- Possible to **invoke the Reduce more than once for the same key.** (In this case, the previous output from the reduce function for that key will become one of the input values to the next reduce function invocation for that key.)

https://tinyurl.com/3jry4azj

Aggregation: mapReduce

Out Options

- Output to a new collection
  - `out: <collectionName>`
- Output to a collection with an Action
  - Action: `replace, merge, reduce`

  - `replace :` Replace the contents of the <collectionName> if the collection with the <collectionName> exists.

  - `merge:` Merge the new result with the existing result if the output collection already exists. If an existing document has the same key as the new result, *overwrite* that existing document.

  - `reduce:` Merge the new result with the existing result if the output collection already exists. If an existing document has the same key as the new result, apply the reduce function to both the new and the existing documents and overwrite the existing document with the result.

https://docs.mongodb.com/manual/reference/command/mapReduce/#dbcmd.mapReduce

MapReduce with Sharded Collections

- MapReduce functions in MongoDB **support operations on sharded Collections**, both as input and output

- **Sharded collection as input:**
  - MongoDB will automatically dispatch the MapReduce job to each shard in parallel

- **Sharded collection as output:**
  - If the out field of the MapReduce function contains the sharded value, MongoDB shards the output collection using the _id field as the shard key automatically

Reduce-Side Join: Data Loading

```
db.createCollection("employee");
db.employee.insert(
 {
  _id: A,
  name: {first:'John',
         last:'Backus'},
  city: 'New York',
  department_id:1
 }
);
db.employee.insert(
 {
  _id: B,
  name: {first:'Merry',
         last:'Desuja'},
  city: 'London',
  department_id:2
 }
);
```

```
db.createCollection("department");

db.department.insert(
 {
  _id: 1,
  department: 'Manager'
 }
);

db.department.insert(
 {
  _id: 2,
  department: 'Accountant'
 }
);
```

Reduce-Side Join: Two Map Functions

```
var mapEmployee = function () {
  var output = {department_id:this.department_id,
                firstname:this.name.first,
                lastname:this.name.last,
                department:null}
  emit(this.department_id, output);
};
var mapDepartment = function () {
  var output = {department_id:this._id,
                firstname:null,
                lastname:null,
                department:this.department}
  emit(this._id, output);
};
```

- We define two functions, each of which will **emit one separate document** with partial data from the Employee and Department collections
- Partial documents with the **same key arrive at the same reducer**.

Reduce-Side Join: Reduce Function

```
var reduceFunction = function(key, values) {
  var outs = {firstname:null, lastname:null , department:null};
  values.forEach(function(v){
         if(outs.firstname == null){
           outs.firstname = v.firstname
         }
         if(outs.lastname == null){
           outs.lastname = v.lastname
    }
    if(outs.department == null){
          outs.department = v.department
    }
  });
  return outs;
};
```

- The Reducer obtains, under each distinct key, a list of documents with the partial data from the Employee and Department documents and merges them into one final output document.

Reduce-Side Join: Execution

```
db.employee.mapReduce(mapEmployee, reduceFunction, {out: {reduce: 'emp_dept'}})

db.emp_dept.find()

db.department.mapReduce(mapDepartment, reduceFunction, {out: {reduce: 'emp_dept'}})

db.emp_dept.find()
```

- Finally, we may execute the Reduce-Side Join by invoking `mapReduce(…)` on the `db.employee` and `db.department` collections using the two Map functions separately.
- **Notice the trick:** by the second invocation of the Reduce function, the value of each Reducer is extended by also the documents from the second collection.